

StatProg2 – Practical 2: Functions

题目、答案与知识点整理 / Aufgaben, Lösungen und Konzepte

目录

1	Exercise 1: Naming Functions	2
1.1	Task 1: 给函数重新命名	2
1.2	Task 2: <code>rnorm()</code> oder <code>norm_r()</code> ?	3
2	Exercise 2: Funktionen schreiben und testen	4
2.1	Task 1: Penguin-Plot in eine Funktion verpacken	4
2.2	Task 2: Docstrings mit <code>roxygen2</code>	6
2.3	Task 3: Randfälle testen	7
2.4	Task 4: Hilfsfunktionen verwenden	8
3	Exercise 3: Input Validation	9
3.1	Task 1: Eingaben prüfen	9
3.2	Task 2: Alte und neue Fehlermeldungen vergleichen	11
4	Exercise 4: Iteration und Rekursion	11
4.1	Task 1: Fibonacci iterativ	11
4.2	Task 2: Fibonacci rekursiv	12
4.3	Task 3: Laufzeit vergleichen	13
5	Exercise 5: Refactoring	14
5.1	Task 1.1: Wiederholungen in <code>summarise_species()</code>	14
5.2	Task 1.2: <code>summarise_species()</code> refaktorisieren	14
5.3	Task 2: <code>penguin_report()</code> in Hilfsfunktionen zerlegen	16
6	Zusammenfassung der wichtigsten Konzepte	18

1 Exercise 1: Naming Functions

1.1 Task 1: 给函数重新命名

给定函数:

```
1 f1 <- function(string, prefix) {  
2   str_sub(string, 1, str_length(prefix)) == prefix  
3 }  
4  
5 f3 <- function(x, y) {  
6   rep(y, length.out = length(x))  
7 }
```

答案

第一个函数检查一个字符串是否以指定前缀开始, 因此可以命名为:

```
1 has_prefix <- function(string, prefix) {  
2   str_sub(string, 1, str_length(prefix)) == prefix  
3 }
```

第二个函数把 y 重复到与 x 相同的长度, 因此可以命名为:

```
1 recycle_to_length <- function(x, y) {  
2   rep(y, length.out = length(x))  
3 }
```

也可以使用更明确的参数名:

```
1 recycle_to_length <- function(value, template) {  
2   rep(value, length.out = length(template))  
3 }
```

知识点

- 好的函数名通常使用动词或动词短语, 例如 `has_prefix()`、`calculate_mean()`。
- 参数名应该说明参数的角色。与 `x`、`y` 相比, `string`、`prefix` 更容易理解。
- R 项目中通常使用 `snake_case`。

Deutsch

Die erste Funktion prüft, ob ein String mit einem bestimmten Präfix beginnt. Ein sinnvoller Name ist daher `has_prefix()`.

Die zweite Funktion recycelt einen Wert oder Vektor, bis die Länge eines Referenzvektors erreicht ist. Ein geeigneter Name ist `recycle_to_length()`.

1.2 Task 2: `rnorm()` oder `norm_r()`?

R verwendet für Wahrscheinlichkeitsverteilungen ein einheitliches Schema:

Präfix	Bedeutung	Normalverteilung
r	Zufallswerte erzeugen	<code>rnorm()</code>
d	Dichte	<code>dnorm()</code>
p	kumulative Wahrscheinlichkeit	<code>pnorm()</code>
q	Quantil	<code>qnorm()</code>

Warum könnte `norm_r()` besser sein?

- Alle Funktionen der Normalverteilung würden mit `norm_` beginnen.
- Durch Autovervollständigung wären `norm_r()`, `norm_d()`, `norm_p()` und `norm_q()` gemeinsam sichtbar.

Warum ist `rnorm()` trotzdem sinnvoller?

- Die Namen folgen einer einheitlichen R-Konvention.
- Das gleiche Schema gilt für andere Verteilungen:

```

1 rbinom()
2 dbinom()
3 pbinom()
4 qbinom()
5
6 runif()
7 dunif()
8 punif()
9 qunif()

```

Wenn man die Präfixe `r`, `d`, `p` und `q` kennt, kann man viele Funktionsnamen vorhersagen.

Noch explizitere Namen wären:

```
1 normal_sample()
2 normal_density()
3 normal_cdf()
4 normal_quantile()
```

2 Exercise 2: Funktionen schreiben und testen

2.1 Task 1: Penguin-Plot in eine Funktion verpacken

Lösung

```
1 library(palmerpenguins)
2 library(tidyverse)
3
4 make_label <- function(variable_name) {
5   str_replace_all(variable_name, "_", " ") %>%
6   str_to_title()
7 }
8
9 scatter_penguins <- function(
10   species_name,
11   x_var = "bill_length_mm",
12   y_var = "bill_depth_mm"
13 ) {
14   penguins %>%
15     filter(species == .env$species_name) %>%
16     ggplot(
17       aes(
18         x = .data[[x_var]],
19         y = .data[[y_var]]
20       )
21     ) +
22     geom_point() +
23     labs(
24       title = paste(
25         "Scatter plot of",
26         make_label(x_var),
27         "vs",
28         make_label(y_var),
29         "for",
30         species_name,
31         "penguins"
32       ),
```

```
33     x = make_label(x_var),  
34     y = make_label(y_var)  
35 )  
36 }
```

Aufruf:

```
1 scatter_penguins("Adelie")  
2  
3 scatter_penguins(  
4   "Gentoo",  
5   "flipper_length_mm",  
6   "body_mass_g"  
7 )
```

关键知识点

- `species` 是数据框中的列。
- `.env$species_name` 表示函数环境中的参数。
- `.data[[x_var]]` 根据字符串形式的列名访问数据列。
- R 函数默认返回最后一个表达式，因此不一定需要 `return(plot)`。

常见错误

错误写法:

```
1 aes(y = .data[[y = y_var]])
```

正确写法:

```
1 aes(y = .data[[y_var]])
```

`.env` 只能在 `filter()`、`mutate()`、`aes()` 等数据掩码环境中使用。在普通的 `paste()` 中应直接写:

```
1 paste(..., species_name, ...)
```

另外, "Torgersen" 是 `island` 的取值, 不是 `species`。企鹅物种是:

```
1 Adelie  
2 Chinstrap  
3 Gentoo
```

2.2 Task 2: Docstrings mit roxygen2

```

1 #' Convert a variable name into a readable label
2 #'
3 #' Replaces underscores with spaces and converts the string
4 #' to title case.
5 #'
6 #' @param variable_name A character string containing a
7 #' variable name.
8 #' @return A readable character string.
9 #'
10 #' @examples
11 #' make_label("bill_length_mm")
12 make_label <- function(variable_name) {
13   str_replace_all(variable_name, "_", " ") %>%
14     str_to_title()
15 }
16
17
18 #' Create a scatter plot for one penguin species
19 #'
20 #' @param species_name A penguin species. Expected values are
21 #' "Adelie", "Chinstrap", and "Gentoo".
22 #' @param x_var Name of the numeric x-axis variable.
23 #' @param y_var Name of the numeric y-axis variable.
24 #' @return A ggplot object.
25 #'
26 #' @examples
27 #' scatter_penguins("Adelie")
28 #' scatter_penguins(
29 #'   "Gentoo",
30 #'   "flipper_length_mm",
31 #'   "body_mass_g"
32 #' )
33 scatter_penguins <- function(
34   species_name,
35   x_var = "bill_length_mm",
36   y_var = "bill_depth_mm"
37 ) {
38   penguins %>%
39     filter(species == .env$species_name) %>%
40     ggplot(
41       aes(
42         x = .data[[x_var]],
43         y = .data[[y_var]]
44       )

```

```

45 ) +
46 geom_point() +
47 labs(
48   title = paste(
49     make_label(x_var),
50     "vs.",
51     make_label(y_var),
52     "for",
53     species_name,
54     "penguins"
55   ),
56   x = make_label(x_var),
57   y = make_label(y_var)
58 )
59 }

```

Deutsch

Roxygen2-Dokumentation beginnt mit `#'`. `@param` beschreibt Argumente, `@return` den Rückgabewert und `@examples` typische Aufrufe.

2.3 Task 3: Randfälle testen

```

1 scatter_penguins("Penguin")
2 scatter_penguins("Adelie", x_var = "fluffiness")
3 scatter_penguins("Adelie", x_var = "island")

```

Test	Verhalten vor Validierung	Problem
Ungültige Art	leerer Plot, meist kein Fehler	stiller Fehler
Nicht vorhandene Spalte	Fehler aus <code>ggplot2</code>	Fehlermeldung ist relativ technisch
Nicht numerische Spalte	Plot mit diskreter Achse	technisch gültig, aber gegen die Funktionsidee

中文说明

- 不存在的物种产生 0 行数据，因此通常只得到空图。
- 不存在的列会在 `ggplot2` 计算映射时出错。
- 分类变量可以被 `ggplot2` 当作离散轴，因此不一定报错。

2.4 Task 4: Hilfsfunktionen verwenden

```
1 make_label <- function(variable_name) {
2   str_replace_all(variable_name, "_", " ") %>%
3   str_to_title()
4 }
5
6 filter_penguin_species <- function(data, species_name) {
7   data %>%
8   filter(species == .env$species_name)
9 }
10
11 scatter_plot <- function(data, x_var, y_var, title) {
12   ggplot(
13     data,
14     aes(
15       x = .data[[x_var]],
16       y = .data[[y_var]]
17     )
18   ) +
19   geom_point() +
20   labs(
21     title = title,
22     x = make_label(x_var),
23     y = make_label(y_var)
24   )
25 }
26
27 scatter_penguins <- function(
28   species_name,
29   x_var = "bill_length_mm",
30   y_var = "bill_depth_mm"
31 ) {
32   title <- paste(
33     "Scatter plot of",
34     make_label(x_var),
35     "vs",
36     make_label(y_var),
37     "for",
38     species_name,
39     "penguins"
40   )
41
42   penguins %>%
43     filter_penguin_species(species_name) %>%
44     scatter_plot(
```



```

45     x_var = x_var,
46     y_var = y_var,
47     title = title
48   )
49 }

```

知识点

每个辅助函数只负责一个任务：

Funktion	Aufgabe
<code>make_label()</code>	lesbare Achsenbeschriftung
<code>filter_penguin_species()</code>	Daten filtern
<code>scatter_plot()</code>	Scatterplot erstellen
<code>scatter_penguins()</code>	einzelne Schritte koordinieren

3 Exercise 3: Input Validation

3.1 Task 1: Eingaben prüfen

```

1 scatter_penguins <- function(
2   species_name,
3   x_var = "bill_length_mm",
4   y_var = "bill_depth_mm"
5 ) {
6   valid_species <- c("Adelie", "Chinstrap", "Gentoo")
7
8   if (!species_name %in% valid_species) {
9     stop(
10      "'species_name' must be one of: ",
11      paste(valid_species, collapse = ", "),
12      ". You supplied: \"",
13      species_name,
14      "\"."
15    )
16  }
17
18  stopifnot(
19    "'x_var' must be a column of 'penguins' " =
20    x_var %in% names(penguins),
21
22    "'y_var' must be a column of 'penguins' " =
23    y_var %in% names(penguins),

```

```
24
25   "'x_var' must be numeric" =
26     is.numeric(penguins[[x_var]]),
27
28   "'y_var' must be numeric" =
29     is.numeric(penguins[[y_var]])
30 )
31
32 penguins %>%
33   filter(species == .env$species_name) %>%
34   ggplot(
35     aes(
36       x = .data[[x_var]],
37       y = .data[[y_var]]
38     )
39   ) +
40   geom_point() +
41   labs(
42     title = paste(
43       make_label(x_var),
44       "vs.",
45       make_label(y_var),
46       "for",
47       species_name,
48       "penguins"
49     ),
50     x = make_label(x_var),
51     y = make_label(y_var)
52   )
53 }
```

检查顺序

必须先检查列是否存在，再检查列是否为数值：

Spalte vorhanden? → Datentyp numerisch?

Testfälle

```
1 scatter_penguins("Emperor")
2 # Fehler: ungültige Art
3
4 scatter_penguins("Adelie", "fluffiness", "body_mass_g")
```

```
5 # Fehler: x_var ist keine Spalte
6
7 scatter_penguins("Adelie", "island", "body_mass_g")
8 # Fehler: x_var muss numerisch sein
```

3.2 Task 2: Alte und neue Fehlermeldungen vergleichen

Die validierte Version ist leichter zu debuggen:

- Sie scheitert früh (*fail fast*).
- Sie nennt direkt das fehlerhafte Argument.
- Sie verhindert stille Fehler wie leere Plots.
- Sie erklärt, welche Eingaben zulässig sind.

Musterantwort

The validated version is easier to debug because it fails early, uses the names of the function arguments, avoids silent failures, and tells the user how to correct the input.

4 Exercise 4: Iteration und Rekursion

4.1 Task 1: Fibonacci iterativ

Die Fibonacci-Folge ist definiert durch:

$$F(0) = 0, \quad F(1) = 1,$$

und für $n > 1$:

$$F(n) = F(n - 1) + F(n - 2).$$

```
1 fib_iterative <- function(n) {
2   if (n == 0) {
3     return(0)
4   } else if (n == 1) {
5     return(1)
6   }
7
8   a <- 0
```

```
9   b <- 1
10
11   for (i in 2:n) {
12     temp <- a + b
13     a <- b
14     b <- temp
15   }
16
17   return(b)
18 }
```

Beispiele:

```
1 fib_iterative(0)
2 # 0
3
4 fib_iterative(5)
5 # 5
6
7 fib_iterative(10)
8 # 55
```

中文解释

初始时:

$$a = F(0) = 0, \quad b = F(1) = 1.$$

每一轮先计算:

$$\text{temp} = a + b,$$

然后把两个保存值向后移动:

$$a \leftarrow b, \quad b \leftarrow \text{temp}.$$

使用 `temp` 是为了避免旧的 `a` 在加法前被覆盖。

4.2 Task 2: Fibonacci rekursiv

```

1 fib_recursive <- function(n) {
2   if (n == 0) {
3     return(0)
4   } else if (n == 1) {
5     return(1)
6   }
7
8   return(
9     fib_recursive(n - 1) +
10    fib_recursive(n - 2)
11  )
12 }

```

递归函数包含:

1. 基本情况: $n == 0$ 和 $n == 1$;
2. 递归情况: 调用 `fib_recursive(n - 1)` 和 `fib_recursive(n - 2)`。

Für $F(5)$:

$$F(5) = F(4) + F(3) = 3 + 2 = 5.$$

Ohne Basisfälle würde sich die Funktion unbegrenzt weiter aufrufen.

4.3 Task 3: Laufzeit vergleichen

```

1 system.time(fib_iterative(30))
2 system.time(fib_recursive(30))

```

Beide Funktionen liefern:

$$F(30) = 832040.$$

Die iterative Variante ist aber wesentlich schneller.

Eigenschaft	Iterativ	Naiv rekursiv
Berechnung	jedes Ergebnis einmal	viele Wiederholungen
Laufzeit	$O(n)$	ungefähr $O(\varphi^n)$
Zusatzspeicher	$O(1)$	Aufrufstapel $O(n)$
Lesbarkeit	etwas länger	nahe an der Formel

Die Rekursion ist nicht grundsätzlich langsam. Das Problem ist hier, dass dieselben Teilprobleme mehrfach berechnet werden. Memoization könnte diese Wiederholungen vermeiden.

5 Exercise 5: Refactoring

5.1 Task 1.1: Wiederholungen in `summarise_species()`

Die ursprüngliche Funktion wiederholt:

- die Filterbedingung für jede Art,
- die Berechnung von Mittelwerten,
- `mean(..., na.rm = TRUE)`,
- die Ausgabe mit `cat()`,
- lange `if/else if`-Zweige.

Trotz ihrer Länge enthält sie nur drei Ideen:

1. auf eine Art filtern,
2. mittlere Schnabellänge und Körpermasse berechnen,
3. Ergebnisse ausgeben.

5.2 Task 1.2: `summarise_species()` refaktorisieren

```
1 summarise_species <- function(  
2   species_name,  
3   data = penguins  
4 ) {  
5   summary_row <- data %>%  
6     filter(species == .env$species_name) %>%  
7     summarise(  
8       mean_bill = mean(  
9         bill_length_mm,  
10        na.rm = TRUE  
11      ),  
12       mean_mass = mean(  
13         body_mass_g,  
14         na.rm = TRUE  
15      )  
16    )  
17 }
```

```

18   cat(
19     species_name,
20     "-- mean bill:",
21     summary_row$mean_bill,
22     "mean mass:",
23     summary_row$mean_mass,
24     "\n"
25   )
26 }

```

Aufrufe:

```

1 summarise_species("Adelie")
2 summarise_species("Gentoo")
3 summarise_species("Chinstrap")

```

Warum funktioniert eine vierte Art?

Die Funktion enthält keine fest codierten Berechnungen für einzelne Arten. Sie verwendet direkt `species_name`. Wenn der Datensatz später eine weitere Art enthält, kann sie ohne neuen `else if`-Zweig verarbeitet werden.

Robustere Variante:

```

1 summarise_species <- function(
2   species_name,
3   data = penguins
4 ) {
5   valid_species <- unique(as.character(data$species))
6
7   if (!species_name %in% valid_species) {
8     stop(
9       "'species_name' must be one of: ",
10      paste(valid_species, collapse = ", ")
11    )
12  }
13
14  summary_row <- data %>%
15    filter(species == .env$species_name) %>%
16    summarise(
17      mean_bill = mean(
18        bill_length_mm,
19        na.rm = TRUE
20      ),
21      mean_mass = mean(
22        body_mass_g,

```

```
23     na.rm = TRUE
24   )
25 )
26
27 cat(
28   species_name,
29   "-- mean bill:",
30   summary_row$mean_bill,
31   "mean mass:",
32   summary_row$mean_mass,
33   "\n"
34 )
35
36 invisible(summary_row)
37 }
```

5.3 Task 2: penguin_report() in Hilfsfunktionen zerlegen

```
1 load_penguins <- function() {
2   palmerpenguins::penguins
3 }
4
5 drop_missing <- function(data, cols) {
6   data[complete.cases(data[, cols]), ]
7 }
8
9 print_summary <- function(data) {
10  cat("n =", nrow(data), "\n")
11  cat(
12    "mean bill length:",
13    mean(data$bill_length_mm),
14    "\n"
15  )
16  cat(
17    "mean body mass:",
18    mean(data$body_mass_g),
19    "\n"
20  )
21 }
22
23 scatter_bill_vs_mass <- function(data, title) {
24   ggplot(
25     data,
26     aes(
27       x = bill_length_mm,
```



```
28     y = body_mass_g
29   )
30 ) +
31   geom_point() +
32   labs(title = title)
33 }
34
35 penguin_report <- function(species_name) {
36   data <- load_penguins() %>%
37     drop_missing(
38       c(
39         "bill_length_mm",
40         "body_mass_g"
41       )
42     ) %>%
43     filter(species == .env$species_name)
44
45   print_summary(data)
46
47   scatter_bill_vs_mass(
48     data,
49     title = paste(
50       "Bill length vs. body mass --",
51       species_name
52     )
53   )
54 }
```

Aufruf:

```
1 penguin_report("Gentoo")
```

Vorteile der Refaktorisierung

- Jede Funktion übernimmt nur eine Aufgabe.
- Einzelne Schritte können separat getestet werden.
- Reinigungs- und Plotfunktionen sind wiederverwendbar.
- Die Hauptfunktion liest sich wie ein Rezept:

laden → bereinigen → filtern → zusammenfassen → visualisieren.

6 Zusammenfassung der wichtigsten Konzepte

1. Funktionen sollten klare Namen und verständliche Parameter besitzen.
2. Wiederholte Codeblöcke sollten durch Funktionen ersetzt werden.
3. Veränderliche Teile gehören in Funktionsargumente.
4. `.data[[name]]` greift auf eine Spalte zu, deren Name als String gespeichert ist.
5. `.env$name` greift innerhalb eines Data-Masking-Kontexts auf ein Funktionsargument zu.
6. Eingaben sollten früh geprüft werden.
7. Gute Fehlermeldungen nennen das Argument, das Problem und möglichst die erlaubten Werte.
8. Rekursion benötigt einen Basisfall.
9. Naive Fibonacci-Rekursion ist wegen mehrfacher Berechnung langsam.
10. Große Funktionen sollten in kleine Hilfsfunktionen mit klarer Verantwortung zerlegt werden.